Программная инженерия

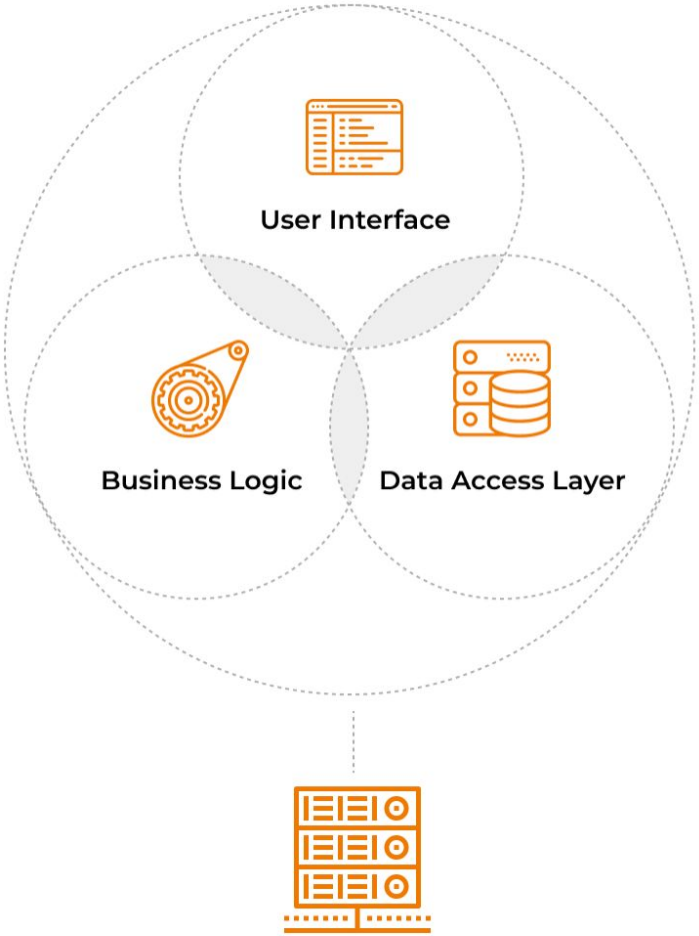
Лекция 9: Сетевые коммуникации между сервисами

Денисов Артём Владимирович

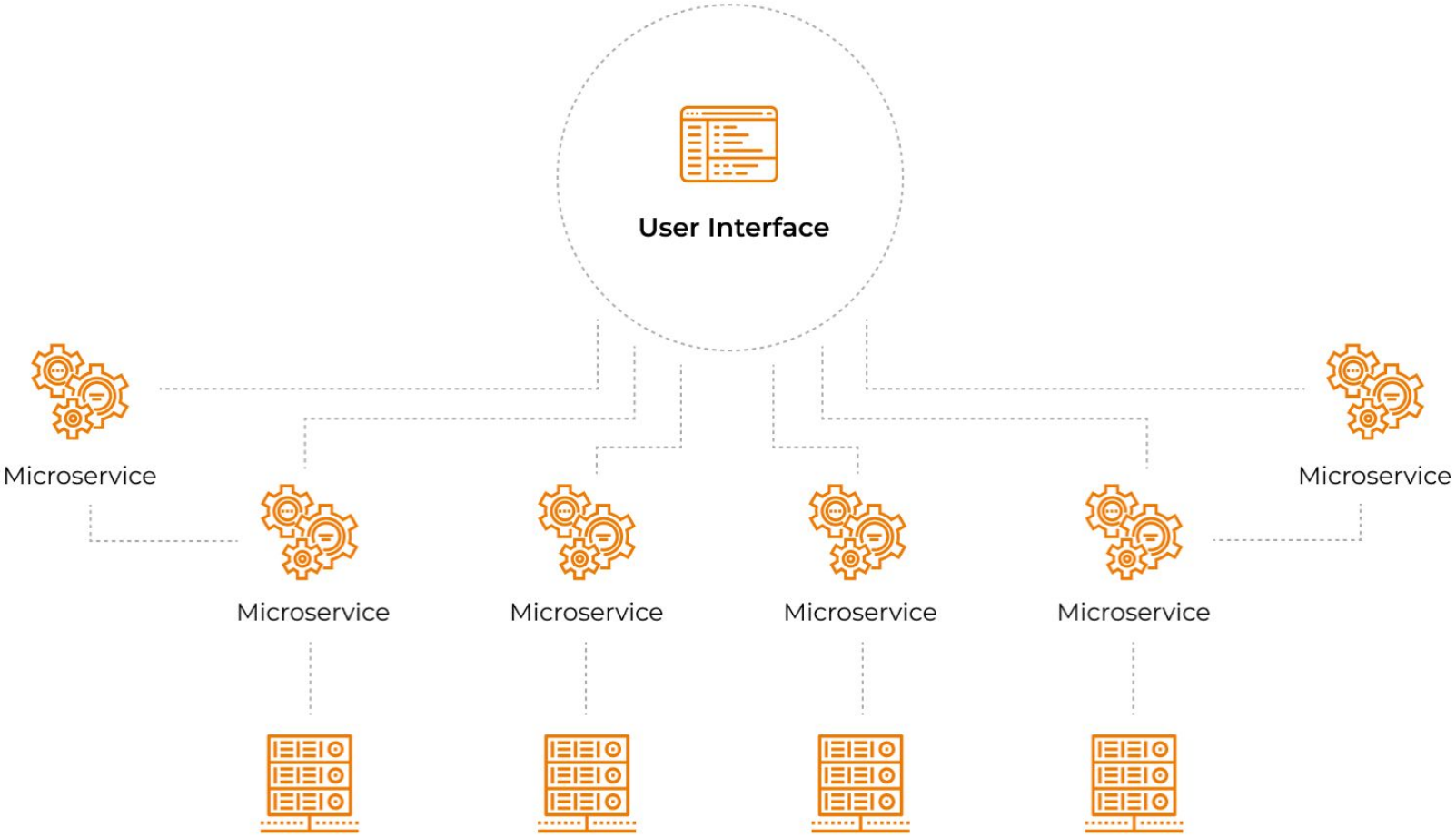
Уральский Федеральный Университет

2026

Монолитная архитектура

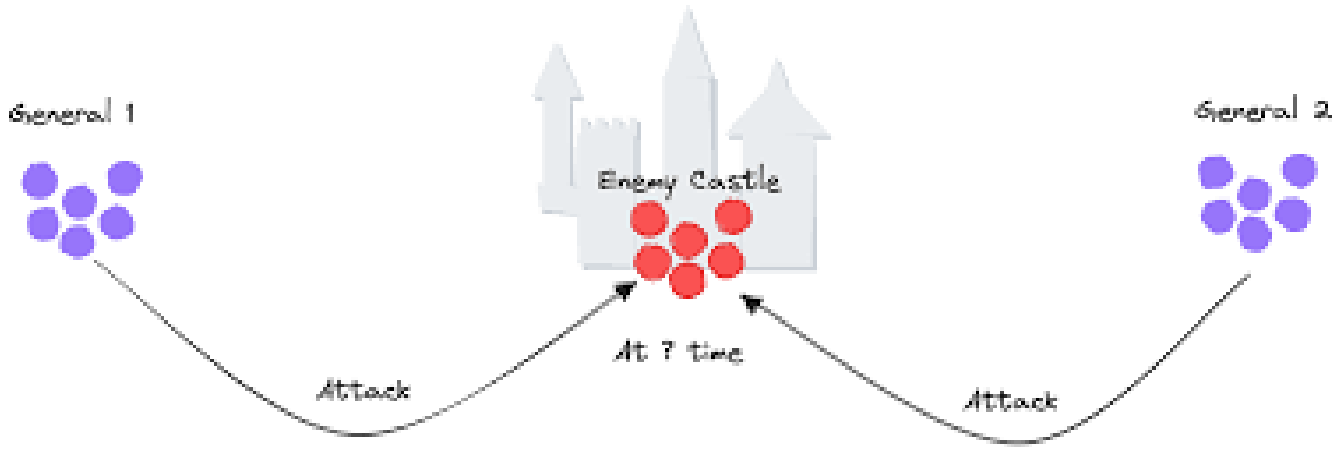


Микросервисная архитектура



Как сервисам общаться друг с другом?

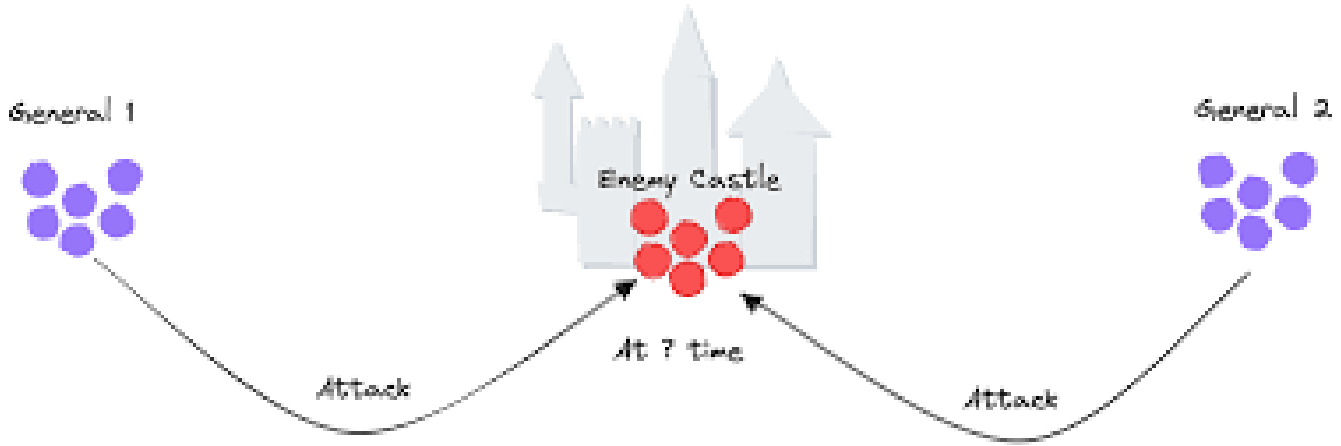
Задача двух генералов



Нужно атаковать замок одновременно

- Лагеря двух армий располагаются на холмах, разделённых долиной
- Единственный способ связи между генералами – отправка посыльных с сообщениями через долину
- Посыльные могут быть перехвачены врагом

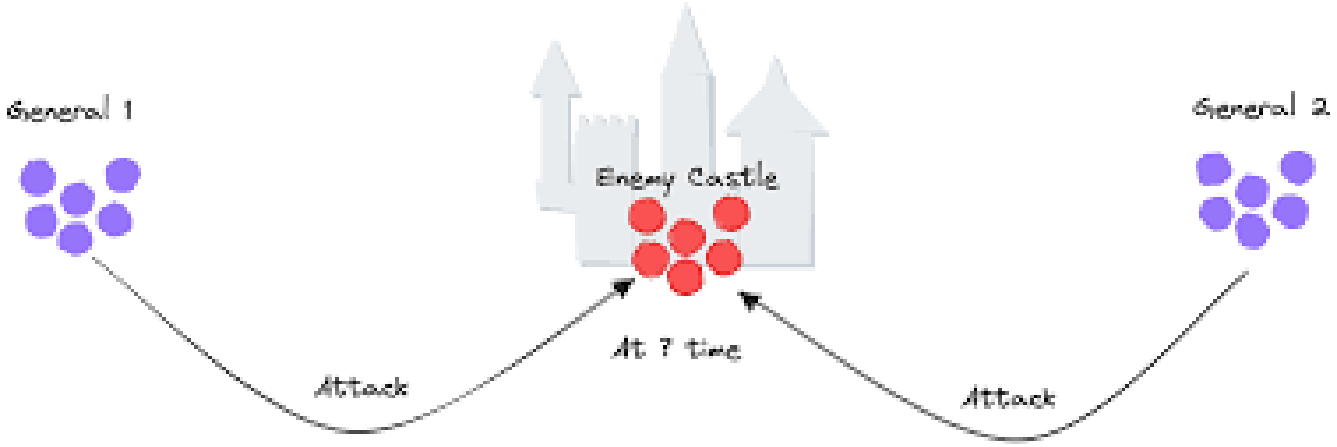
Задача двух генералов



Как организовать одновременное наступление?

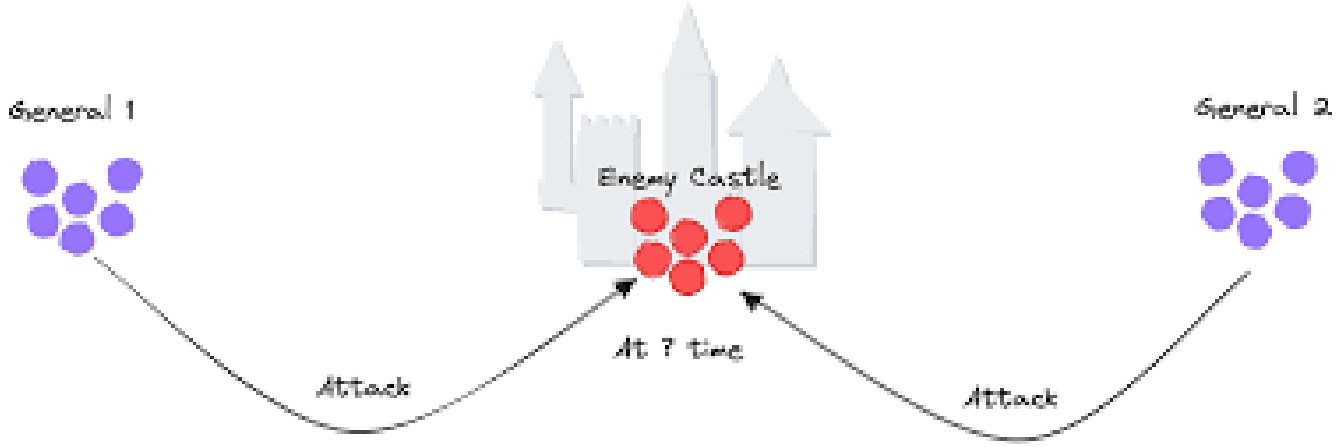
- Для успешного штурма генералы должны атаковать город одновременно
- Штурм, предпринятый только одной армией - поражение

Задача двух генералов



Возможно ли разработать алгоритм гарантированного обмена этими сообщениями?

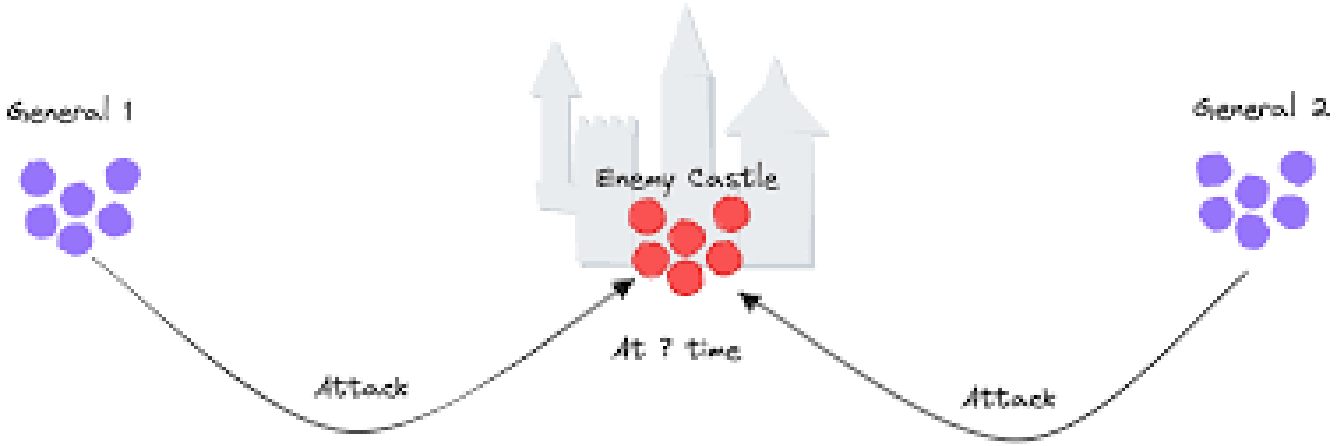
Задача двух генералов



Возможно ли разработать алгоритм гарантированного обмена этими сообщениями?

Нет

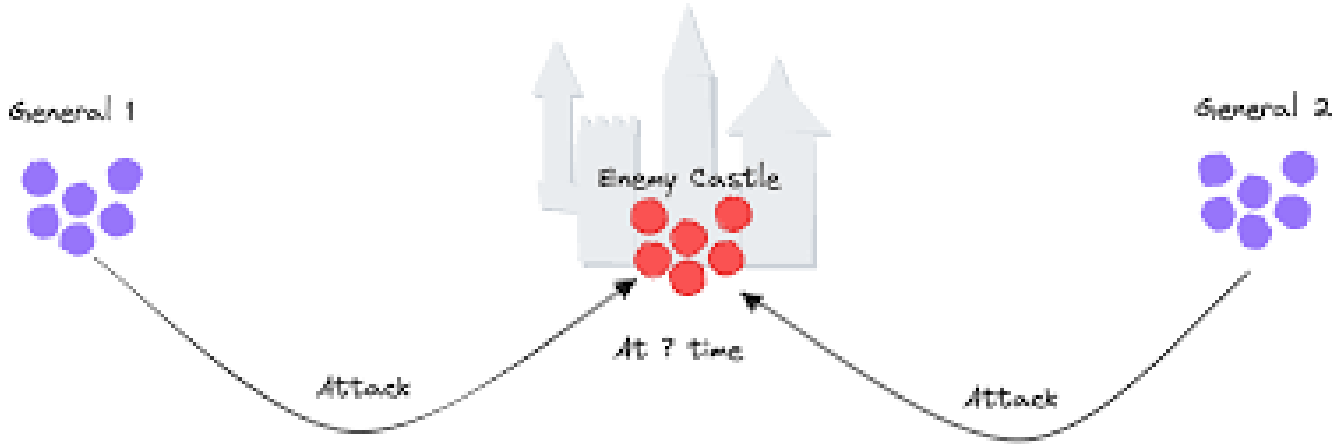
Задача двух генералов



Инженерные подходы

- Отправить 100 посыльных

Задача двух генералов



Инженерные подходы

- Отправить 100 посыльных
- Отправлять посыльных до тех пор, пока не поступит ответ-подтверждение
- Если первый получил подтверждение, он должен подтвердить подтверждение. Но и это сообщение может потеряться.

Сетевые коммуникации между сервисами

Что меняется, когда сервисы общаются по сети?

- Появляется задержка и неопределённость
- Сбои становятся ожидаемым сценарием

Главная мысль

- Сеть делает вызов ненадёжным
- Протоколы — это набор инженерных компромиссов

Почему протоколов так много?

Проблемы

- Нагруженность
- Масштабируемость
- Отказоустойчивость
- Поддержка

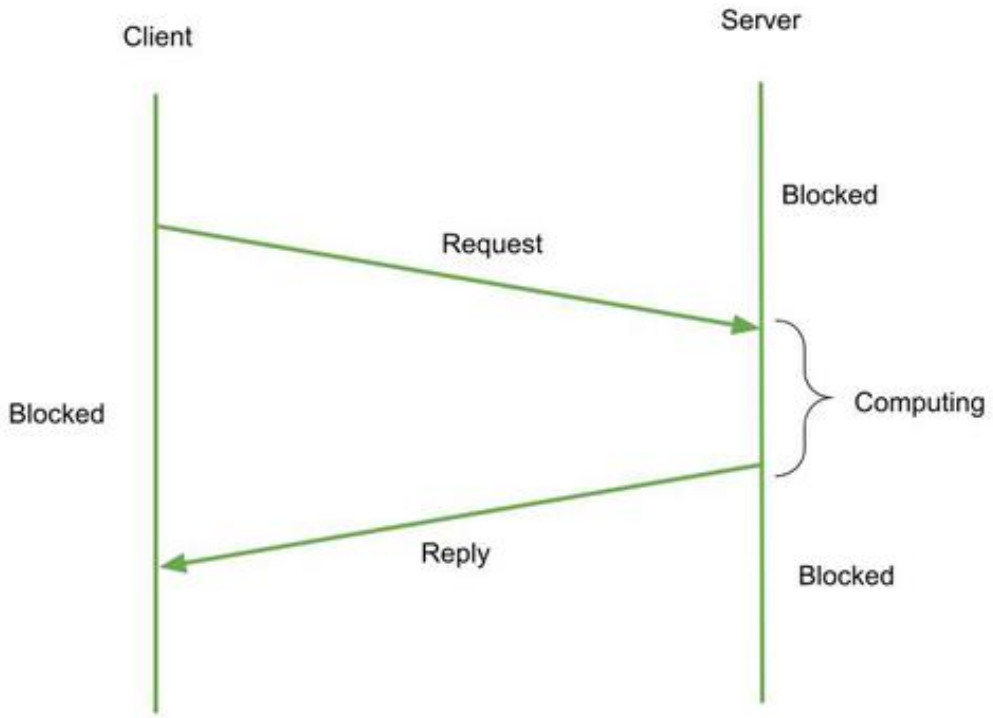
Нет одного лучшего протокола для всех задач

- Каждый протокол фиксирует свой компромисс

Три базовых класса

- Синхронные: HTTP, gRPC, GraphQL
- Асинхронные реактивные: WebSocket, SSE
- Асинхронные событийные: AMQP, Kafka, MQTT

Синхронное взаимодействие



Что это?

- Клиент отправляет запрос и ждёт ответ
- Модель мышления: “request — response”

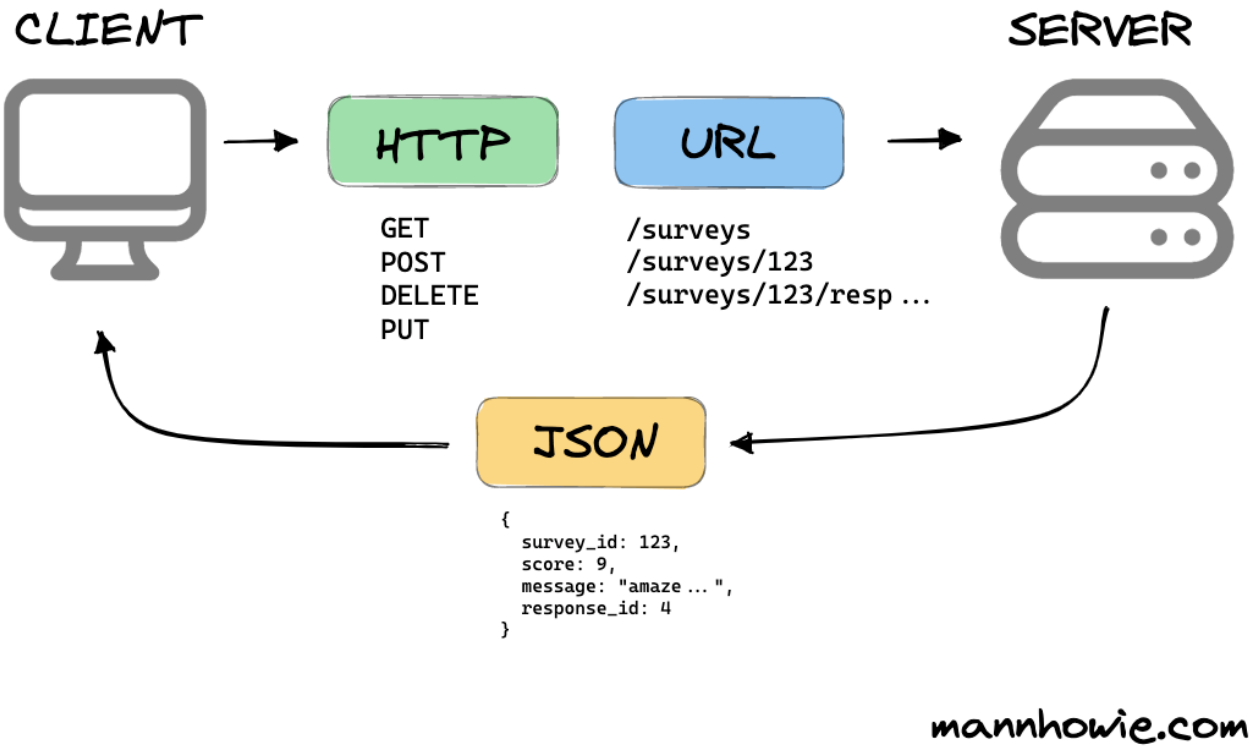
REST API

HTTP / REST API

Что это?

- HTTP — это семейство stateless application-level request/response протоколов
- REST — это способ организовать API поверх HTTP
- типичный формат данных — JSON

REST API


CLIENT
SERVER
HTTP
URL
GET
POST
DELETE
PUT
/surveys
/surveys/123
/surveys/123/resp ...
JSON
{
survey_id: 123,
score: 9,
message: "amaze ... ",
response_id: 4
}
mannhowie.com

REST API: сильные стороны и ограничения

Почему REST популярен

- Совместим со всей web-инфраструктурой
- Низкий порог входа
- Данные передаются в текстовом формате
- Легко читать и отлаживать

Где начинаются проблемы

- Неэффективный способ передачи данных
- Лишние запросы
- Слабо формализован контракт
- Соблазн использовать “везде”

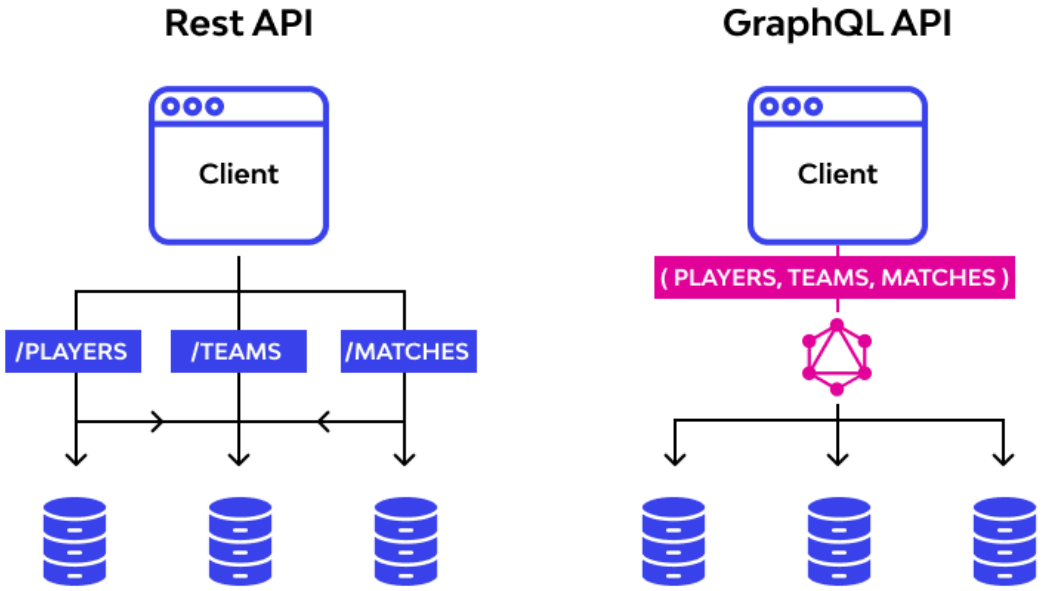
Когда REST — хороший выбор

Подходит

- Как внешний вход в систему
- Публичный API
- CRUD-интерфейс
- Простые интеграции

GraphQL

Как работает GraphQL



Базовый цикл

- Сервер публикует схему
- Клиент отправляет query запрос
- Сервер исполняют запрос, отправляет только нужные поля

GraphQL vs REST

REST

- Много endpoint-ов
- Сервер задаёт форму ответа
- Часто есть лишние данные

GraphQL

- Часто один endpoint
- Клиент задаёт форму ответа
- Гибкие выборки. Запрашиваются нужные поля

Где GraphQL особенно уместен

Стоит рассмотреть, если

- У клиентов разные потребности в данных
- UI собирает данные из нескольких сущностей
- Важна гибкость выборки полей

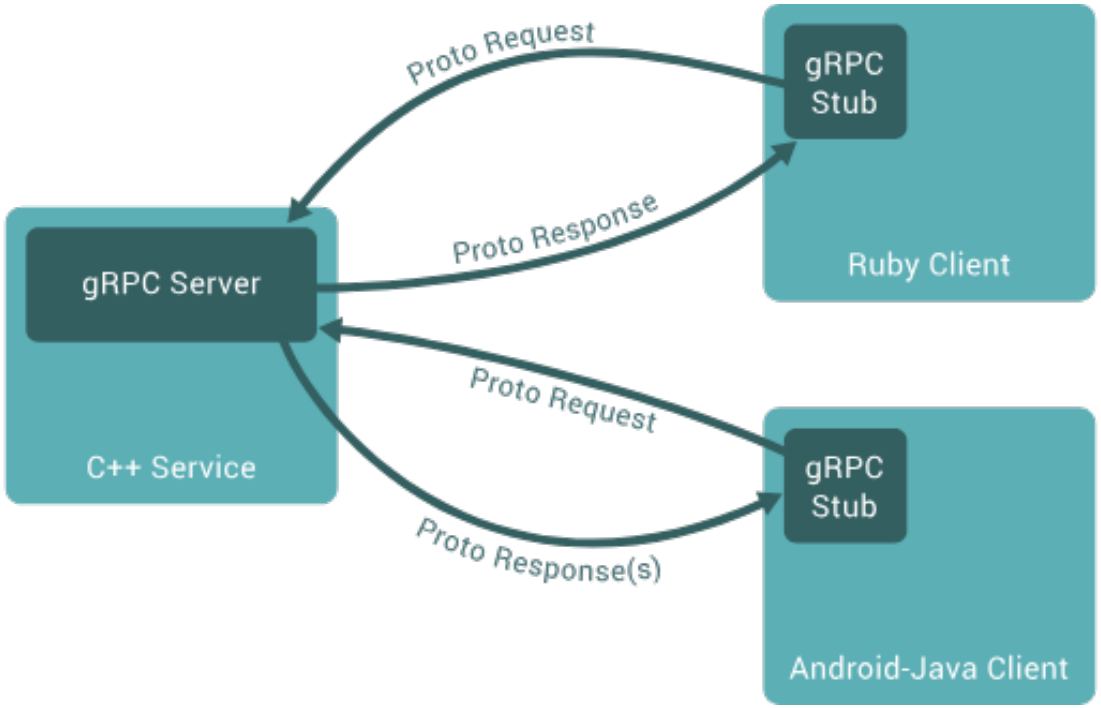
RPC

RPC и gRPC

RPC - Remote Procedure Call

- Вызов удалённой операции как функции
- обычно использует HTTP/2 и Protocol Buffers
- gRPC: современный open source RPC-фреймворк от Google

Как работает gRPC



Особенности

- Объекты и вызовы описываются в `.proto`
- Клиент вызывает метод, как свой собственный
- Сервер возвращает результат метода

Protocol Buffers

```
syntax = "proto3";

message User {
  int32 id = 1;
  string name = 2;
  string email = 3;
}
```

Алогритм действий

1. Вы описываете структуру сообщения в файле .proto
2. Компилятор protoc генерирует классы или структуры для нужного языка
3. Данные сериализуются в компактный бинарный вид
4. На другой стороне они десериализуются обратно в объект

gRPC: плюсы и риски

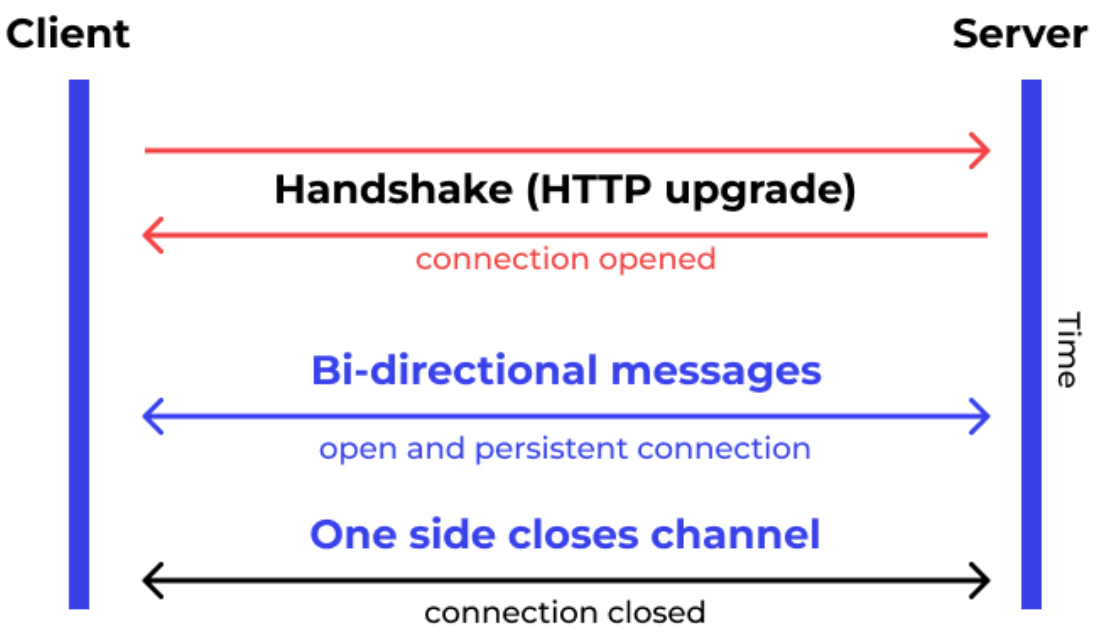
Сильные стороны

- строгий контракт
- компактная сериализация (binary)

Главные риски

- Выше порог входа
- Хуже читается без инструментов
- Неестественен для публичного web API
- В коде не отличается от несетевых вызовов

Асинхронное реактивное взаимодействие

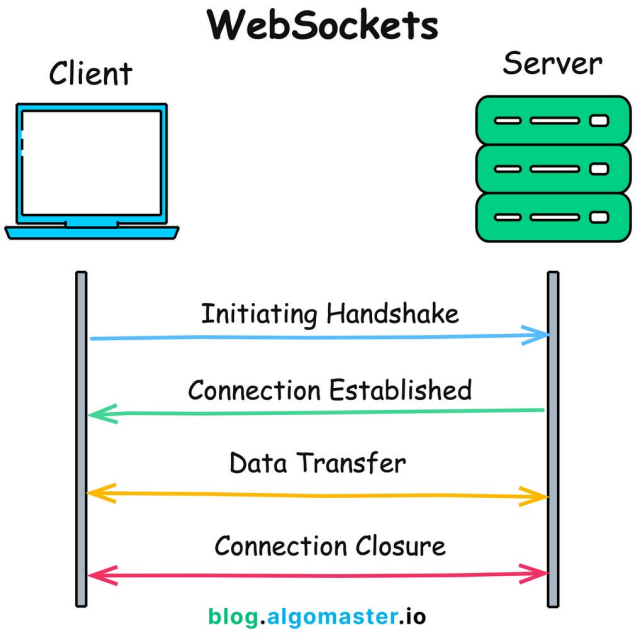


Особенности

- Устанавливается долгоживущий двунаправленный или однонаправленный канал
- Данные передаются в реальном времени по потокам

WebSocket

Как работает WebSocket



Базовый цикл

- Клиент начинает HTTP-like handshake
- Сервер подтверждает upgrade
- Соединение остаётся открытым
- Обе стороны шлют сообщения независимо

WebSocket: сильные стороны и ограничения

Плюсы

- Двусторонняя связь
- Низкая задержка после установки
- Нет постоянного HTTP-overhead

Минусы

- Сложнее в эксплуатации
- Много соединений - больше нагрузка на сервер (в сравнении с HTTP)

Где WebSocket особенно уместен

Типичные сценарии

- Чаты
- Игры
- Live dashboards

WebSocket полезен когда обе стороны активно обмениваются событиями

Socket.IO: Надстройка над WebSocket

Что это?

- Библиотека
- Чаще всего использует WebSocket

Главная мысль

- Socket.IO — это более высокий уровень абстракции

Что даёт Socket.IO разработчику

Удобства из коробки

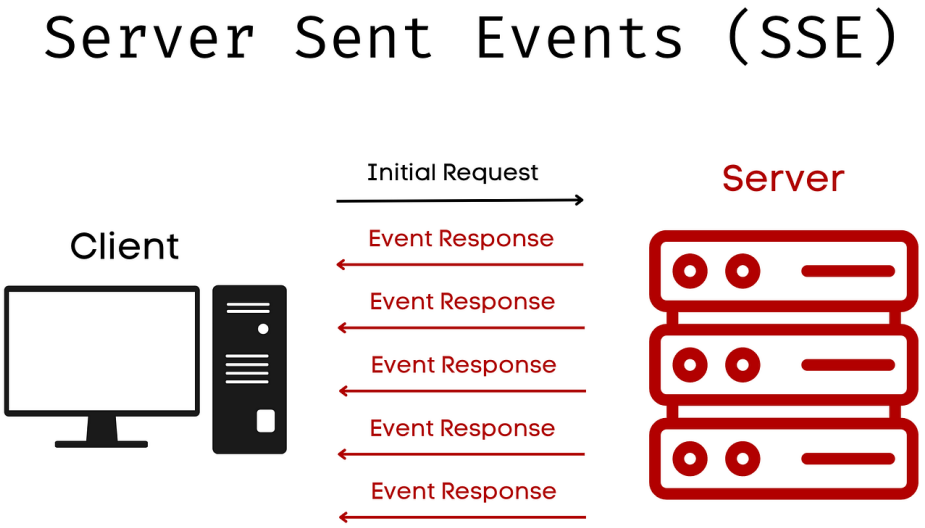
- automatic reconnection
- acknowledgments
- rooms и namespaces
- broadcasting
- буферизация событий

Компромиссы

- дополнительная зависимость
- несовместим с “чистым” WebSocket API

Server-sent events (SSE)

Как работает SSE



Базовый цикл

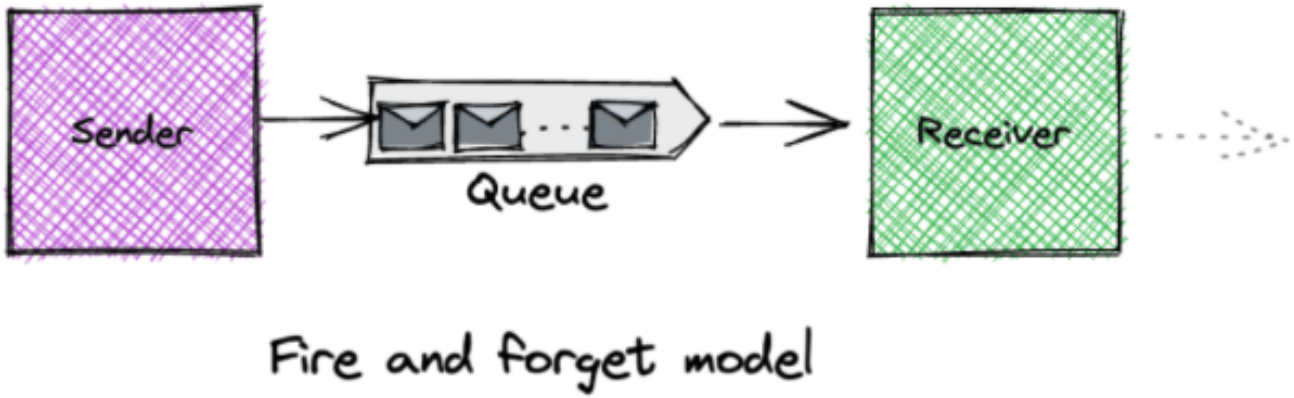
- Сервер пушит события клиенту
- Соединение держится открытым
- Связь идёт только от сервера к клиенту

SSE: server push без двустороннего канала

Когда это удобно

- Уведомления
- Ленты обновлений
- Статус фоновых задач

Асинхронное событийное взаимодействие



Особенности

- Сообщения публикуются в брокер/топик
- Получатели обрабатывают их асинхронно
- Publisher и Consumers может быть несколько

Зачем нужен брокер сообщений

Главная идея

- Сервисы общаются не прямым вызовом
- Сообщение сначала попадает в брокер
- Обработка может произойти позже

Что это даёт

- Асинхронность
- Устойчивость к всплескам нагрузки

Что делает брокер сообщений

Базовая роль

- Принимает сообщения от producer-ов
- Хранит их
- Доставляет по правилам маршрутизации

Почему это важно

- Producer не ждёт consumer-а синхронно
- Consumer может быть недоступен в момент публикации

Event-driven архитектура

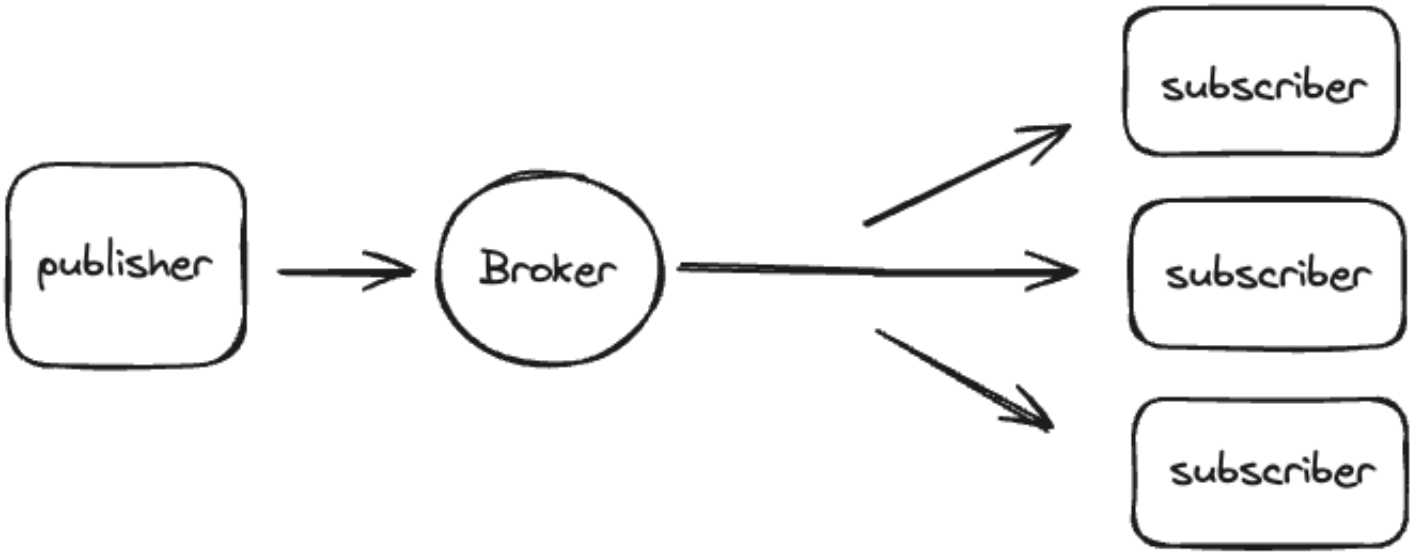
Главная идея

- Сервис сообщает факт (событие)
- Другие сервисы реагируют на событие
- Producer не знает consumer-ов

Что это меняет

- Меньше прямых вызовов по цепочке
- Меньше coupling
- Проще добавлять новых подписчиков на событие

Producer / Consumer / Pub-Sub



Роли

- Producer публикует сообщение
- Broker принимает и маршрутизирует
- Consumer читает и обрабатывает

Почему брокер уменьшает coupling

При HTTP или RPC

- Сервис A знает адрес B
- Ждёт ответ и обрабатывает его сразу

При MQ

- Сервис A знает очередь брокера
- Отправил сообщение и забыл
- Не зависит от consumer-ов

Цена слабого coupling

Что выигрываем

- Легче переживать недоступность сервисов
- Проще подключать новых обработчиков

Что усложняется

- Наблюдаемость потока работы (новая точка отказа)
- Отложенные ошибки

Гарантии доставки

Проблема

- Сообщение может потеряться
- Сообщение может прийти дважды
- Сбой может случиться у broker-а или consumer-а

Поэтому

- система должна явно обеспечивать гарантию доставки

At most once и at least once

At most once

- Не более одной доставки
- Дубликатов нет
- Сообщение может потеряться

Интуиция

- Лучше потерять, чем повторить

At least once

- Как минимум одна доставка
- Потеря менее вероятна
- Возможны дубликаты

Интуиция

- Лучше повторить, чем потерять

Exactly once

Что означает exactly once

Даже если сообщение отправляется несколько раз, система гарантирует, что оно будет доставлено и обработано строго один раз

Проблема

Требует сложной логики и реализации на всех этапах (producer, brocker, consumer)

Вывод

Часто выбирают At least once, так как добиться Exactly once технически сложно и накладно по производительности

Когда брокер особенно полезен

Хороший выбор, если

- Не нужен мгновенный ответ пользователю
- Нужна фоновая обработка
- Нескольких consumers
- Нужно переживать пики нагрузки

Когда брокер — плохой выбор

Не лучший вариант, если

- Нужен ответ сразу
- Нужна жёсткая синхронная валидация
- Важна простота потока управления

Выводы по теме

Главная мысль

- В распределённой системе сеть делает вызов **медленным, ненадёжным и частично наблюдаемым**
- Выбор протокола — это выбор **инженерного компромисса**

Практический вывод

- **HTTP / REST** — хороший базовый выбор для простых API и внешних интеграций
- **gRPC** — подходит для внутренних service-to-service вызовов, где важны скорость и строгий контракт
- **SSE / WebSocket** — нужны, когда важен realtime
- **Broker / event-driven подход** — нужен для фоновой обработки, fan-out и устойчивости к пикам нагрузки

Источники информации и изображений

1. https://www.itweek.ru/management/article/detail.php?ID=223826
2. https://www.geeksforgeeks.org/computer-networks/rpc-message-protocol/
3. https://status-code.medium.com/rest-api-principles-3859e7699fd4
4. https://grpc.io/docs/what-is-grpc/introduction/
5. https://www.wallarm.com/what/what-is-graphql-definition-with-example
6. https://medium.com/pon-tech-talk/a-simple-guide-to-server-sent-events-sse-and-eventsource-9de19c23645b
7. https://www.wallarm.com/what/a-simple-explanation-of-what-a-websocket-is
8. https://eda-visuals.boyney.io/visuals/sync-vs-async
9. https://blog.algomaster.io/p/websockets
10. https://www.digitalocean.com/community/tutorials/publish-subscribe-pattern-in-node-js

Photographs used under public domain or fair usage. No copyright infringement intended. If used accidently under wrong license please contact to be removed.

Программная инженерия

Лекция 9: Сетевые коммуникации между сервисами

Денисов Артём Владимирович

Уральский Федеральный Университет

2026